

App Development Project Report

1. Student Details

Name: P N Megarajan

Roll Number: 24f3003242

Email: 24f3003242@ds.study.iitm.ac.in

About Me: I am a student at the IIT Madras BS Data Science Degree Program with a strong interest in full-stack web development and ML and DL. I enjoy building practical web applications and continuously improving my programming and problem-solving skills through real-world projects.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

To develop a web-based Trekking Management Application that enables administrators to manage treks, staff, and users efficiently. The system allows trekkers to browse and book available treks, while trek staff manage assigned treks, helping eliminate manual coordination, prevent overbooking, and maintain complete trekking history.

Approach:

The application was developed using the Flask framework following a role-based access control approach for Admin, Trek Staff, and Users. SQLite was used as the backend database with SQLAlchemy ORM, while the frontend was built using Jinja2, HTML, Bootstrap, and CSS to create a responsive and user-friendly interface. The system includes authentication, trek management, booking management, status tracking, and profile management features.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in understanding Flask concepts, resolving coding issues, and improving the application interface.

The extent of AI/LLM usage is around **10–13%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core Backend web framework
SQLAlchemy	Object Relational Mapper(ORM) for database operations
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend Styling and responsive user interface
Flask-Login	User authentication and session management
SQLite	Lightweight relational database for storing application data

5. Database Schema / ER Diagram

Tables:

1. **User** — Stores user, staff, and admin details (ID, Name, Email, Password, Phone, Role, Approval Status, Blacklist Status).
2. **Trek** — Stores trek information (Trek ID, Trek Name, Location, Difficulty, Duration, Available Slots, Assigned Staff, Status, Start Date, End Date).
3. **Booking** — Stores booking records for trekkers (Booking ID, User ID, Trek ID, Booking Date, Booking Status, Payment Status).

Relationships:

- One-to-Many → User → Booking
- One-to-Many → Trek → Booking
- One-to-Many → User (Staff) → Trek



6. API Resource Endpoints

Endpoint	Method	Description
/	GET	Redirect authenticated users to their dashboard
/register	GET, POST	Register a new trek user or staff
/login	GET, POST	Authenticate user credentials
/logout	GET	Logout the current user
/admin	GET	View admin dashboard
/admin/treks	GET	View and search all treks
/admin/add_trek	GET, POST	Add a new trek
/admin/edit_trek/{trek_id}	GET, POST	Edit trek details
/admin/delete_trek/{trek_id}	GET	Delete a trek
/admin/users	GET	Manage registered users
/admin/staff	GET	Manage trek staff
/admin/bookings	GET	View all booking records
/staff	GET	View staff dashboard
/staff/edit_trek/{trek_id}	GET, POST	Update assigned trek details
/staff/participants/{trek_id}	GET	View participants of assigned trek
/user	GET	View user dashboard
/user/treks	GET	Browse and search available treks
/user/book/{trek_id}	GET	Book an available trek
/user/bookings	GET	View personal booking history
/user/cancel_booking/{booking_id}	GET	Cancel an existing booking

YAML API Definition File:

Included separately in the submission ZIP as api.yaml.

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – Main Flask application entry point that initializes the application, database, authentication, and registers application routes.
- **models.py** – Contains SQLAlchemy database models for User, Trek, and Booking.

- **routes.py** – Implements all application routes and business logic for Admin, Trek Staff, and User functionalities.
- **templates/** – Contains Jinja2 HTML templates used to render dynamic pages for different user roles.
- **static/** – Stores static assets such as CSS files used for styling the application.

Implemented Features:

Authentication

- User and Trek Staff registration
- Secure login and logout using Flask-Login
- Password hashing using Werkzeug Security

Admin Features

- Dashboard with system statistics
- Add, edit, and delete treks
- Approve or blacklist trek staff
- Blacklist or activate users
- Assign staff to treks
- View all bookings
- Search users, staff, and treks

Trek Staff Features

- View assigned treks
- Update available slots
- Update trek status
- View trek participants
- Update staff profile

User Features

- Browse available treks
- Search and filter treks
- Book treks
- Cancel bookings
- View booking history
- Update user profile

Other Features

- Prevent duplicate bookings
- Prevent overbooking beyond available slots
- Allow bookings only for open treks
- Role-based access control
- Responsive user interface using Bootstrap

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1NJT1I-1aHRd9nLI8aKAdoRma3c_Kjuj1/view?usp=sharing